

mqttConnEVT

Overview

`mqttConnEVT` is a **device-emitted** event that reports an MQTT endpoint's **connection-state transition** — `CONNECTED` or `DISCONNECTED` — along with device identity (`deviceModel` , `deviceSerialNo`) and protocol-version metadata (`apiVersion` , `mqttVersion`). The reader publishes it **autonomously** on every connection-state change; **no command triggers it and no configuration enables it**. It is read-only telemetry and carries no credentials.

It behaves like standard MQTT connection lifecycle messages: a `CONNECTED` event is a *birth* message published when an endpoint (re)connects, and a `DISCONNECTED` event is a [Last Will](#) message the broker publishes when the device session drops. **Each active endpoint emits its own `mqttConnEVT`.**

`mqttConnEVT` is published on each endpoint's `{EP}/clients/rfid` topic (e.g. `<tenantId>/MGMT/clients/rfid/<serial>` , `<tenantId>/CTRL/clients/rfid/<serial>`) — **not** the `{EP}/clients/event` topic that carries `alerts` and `heartBeatEVT` , and not the DATA topic that carries `dataEVT` . Subscribe to the `rfid` topic of each endpoint you want connection events for.

Details

Property	Value
Event Type	Connection-state transition (birth / Last Will)
Trigger	Connection transition (on every MQTT connect / disconnect; autonomous, with no command or config)
Applies To	Endpoint: Any endpoint (MGMT , CTRL , DATA). Reader: RFD40 series, RFD90 series
Credentials	None (read-only telemetry, nothing to mask)
Interaction Pattern	Publish / Subscribe (device-emitted, unsolicited; no request/response, no <code>response.code</code>)
Related	<code>alerts</code> , <code>heartBeatEVT</code> , <code>dataEVT</code>

When to Use

Subscribe to `mqttConnEVT` when you need a **reliable, per-endpoint presence/liveness signal** — to learn the moment an MQTT endpoint (re)connects or drops, without polling. The reader emits it **autonomously** on every connection-state change; there is nothing to enable and no command to issue.

- **Get:** each endpoint's connect / disconnect transitions (`connectionState` = `CONNECTED` / `DISCONNECTED`), with device identity (`deviceModel`, `deviceSerialNo`) and protocol-version metadata (`apiVersion`, `mqttVersion`).
- **Use it for:** presence/liveness dashboards, triggering reconnect and re-subscribe logic, and detecting a dropped device even when the device itself cannot report it — the `DISCONNECTED` Last Will is broker-published.
- **Prefer a different approach when:** you want an on-demand, correlated read rather than a push — poll `get_status`, whose reply itself confirms the device is reachable and reports `deviceStatus.radioConnection`. Use the event for continuous presence monitoring and `get_status` for a point-in-time check.

Topic Mapping

`mqttConnEVT` is device-emitted and has no `cmdnd` / `resp` topics. It is published on each active endpoint's own `rfid` topic.

Message	Direction	Topic	Note
Event	Reader → App	<code><tenantId>/MGMT/clients/rfid/<serial></code>	The management endpoint's connection events. <code>CONNECTED</code> is the MQTT birth message; <code>DISCONNECTED</code> is the broker Last Will. Published <code>retain=false</code> — a subscriber that connects after a transition will not see the last state replayed.
Event	Reader → App	<code><tenantId>/CTRL/clients/rfid/<serial></code>	The CTRL endpoint's connection events — each active endpoint emits its own (<code>retain=false</code>).

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Here the `rfid` tail is the endpoint's configured topic name, not tag data. On a reboot-triggered reconnect, each active endpoint (MGMT, CTRL, ...) publishes its own `CONNECTED` event on its own `rfid` topic.

See [Topics and Channels](#) and [Broker Connection](#).

Key Fields

- `connectionState` (event) — `CONNECTED` (birth message: the endpoint (re)connected) or `DISCONNECTED` (Last Will: the session was lost). This is the field you route and act on.
- `timestamp` (event) — ISO-8601 instant of the transition (`format: date-time`). Present on `CONNECTED` ; typically omitted on `DISCONNECTED` , because the device is gone when the broker publishes the Last Will.
- `deviceModel` / `deviceSerialNo` (event) — device identity: model (`RFD40` / `RFD90`) and the unique serial number, so you can attribute each transition to a specific reader.
- `apiVersion` (event) — IOTC API version as a bare JSON number (e.g. `1.21`); note it differs from the `v1.21` string command responses return (see [Notes and Caveats](#)).
- `mqttVersion` (event) — MQTT protocol version (MQTT 3.1.1), serialized as the float `3.09999999` (≈ `3.1` , a lossy JSON-number encoding); treat any value ≈ `3.1` as MQTT 3.1.1.

Event Payload

Six top-level fields; the schema requires all of them except `timestamp` , which may be absent (notably on `DISCONNECTED`). Pick a **connection state** below; the Example and Schema update together. Use the **Schema** tab for the field reference and the **Example** tab for the concrete payload.

Connection state

Connected event



Event payload

Example: `CONNECTED` (live)

```
1 {
2   "connectionState" : "CONNECTED" ,
3   "timestamp" : "2026-06-15T13:09:33.810Z" ,
4   "deviceModel" : "RFD40" ,
5   "deviceSerialNo" : "RFD40-211545201D0034" ,
6   "apiVersion" : 1.21 ,
7   "mqttVersion" : 3.09999999
8 }
```

Event payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	co nn ec ti on St at e *	enum	CONNECTED DISCONNECTED	CONNECTED	Endpoint connection status. CONNECTED = the endpoint (re)connected (birth message); DISCONNECTED = the session was lost (Last Will). See Event Semantics .
2	time stam p	string	date-time	2026-06-15T13:09:33.810Z	ISO-8601 time of the transition (format: date-time). Present on CONNECTED; typically omitted on DISCONNECTED (see Event Semantics).
3	de vi ce Mo de l *	enum	RFD40 RFD90	RFD40	Reader hardware model.
4	de vi ce Se ri al No *	string	—	RFD40-24190525100354	Unique device serial number.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
5	apiVersion *	number	v1.0 v1.1 v1.2 v1.21 v1.22	1.21	IOTC API version, emitted as a bare JSON number (e.g. 1.21). Note this differs from the v1.21 string that command responses return, and the schema pairs type: number with an enum of v... strings that no number can match — see Notes and Caveats .
6	mqttVersion *	number	—	3.0999999	MQTT protocol version — MQTT 3.1.1. Emitted as a JSON number; the device serializes it as the float 3.0999999 (≈ 3.1), a lossy encoding (see Event Semantics).

Event Semantics

States. `connectionState` carries one of two values, each tied to an MQTT lifecycle role:

connectionState	MQTT role	When	timestamp
CONNECTED	Birth message	Published by the endpoint when it (re)connects to the broker.	Present (ISO-8601).
DISCONNECTED	Last Will (LWT)	Published by the broker when the device session drops (detected via keepalive).	Usually omitted — the device is gone, so it cannot stamp it.

Because `DISCONNECTED` is a broker-published Last Will, it can arrive some time after the actual drop, once the broker detects the lost session through the keep-alive, and it carries no `timestamp`. `CONNECTED` events follow on reconnect, each with its own ISO-8601 `timestamp`.

- The device serializes `mqttVersion` as the JSON number `3.0999999`, a lossy floating-point encoding of `3.1`. The MQTT protocol in use is **3.1.1**; treat any `mqttVersion` value ≈ 3.1 as MQTT 3.1.1, and do not test it for exact equality to `3.1`. (This is a known firmware serialization quirk.)
- `timestamp` **is optional and format** `date-time`. When present it is a full ISO-8601 instant (e.g. `2026-06-15T13:09:33.810Z`). It is typically **omitted on** `DISCONNECTED` (a broker Last Will published after the device is gone), which the schema permits: `timestamp` is the one field outside its `required` set.
- Per-endpoint emission**, `retain=false`. Each active endpoint publishes its own `mqttConnEVT` on its own `{EP}/clients/rfid` topic, so a single reconnect yields one event per endpoint (e.g. MGMT and CTRL each publish a `CONNECTED`). All are published with `retain=false`, so a late subscriber will not see the last state replayed — subscribe before the transition you want to observe.

Examples

The **Connection state** dropdown under [Event Payload](#) shows the contract's `CONNECTED` event and its `DISCONNECTED` Last Will, which has no `timestamp`. Each endpoint publishes its own on its own `rfid` topic.

A second `CONNECTED` event on the **CTRL** endpoint's `rfid` topic (per-endpoint emission — the MGMT endpoint published its own moments earlier):

mqttConnEVT — CONNECTED on the CTRL endpoint (live)

```
1 {
2   "connectionState" : "CONNECTED" ,
3   "timestamp" : "2026-06-14T12:54:51.411Z" ,
4   "deviceModel" : "RFD40" ,
5   "deviceSerialNo" : "RFD40-24190525100255" ,
6   "apiVersion" : 1.21 ,
7   "mqttVersion" : 3.0999999
8 }
```

Notes and Caveats

- `apiVersion` is a JSON number here, a `v...` string in command responses. `mqttConnEVT` declares `apiVersion` as a **number**, so the event emits it bare — `"apiVersion": 1.21`. Every command response declares it a **string** with a leading `v` — `get version`, for instance, returns `"apiVersion": "v1.21"`. A shared envelope parser that assumes a string will break on this event (a strict decoder throws; a lenient one silently coerces `1.21` to `"1.21"`, which never matches `"v1.21"`). Parse the event's value as a number, and normalize both forms before you compare them — same version family, two representations. The schema also gives this number an enum of `v...` strings (`v1.0` ... `v1.22`), which no JSON number can equal, so its own examples fail that enum — a known issue in the source schema. Validate the type, not the enum.
- No response envelope; automatic.** `mqttConnEVT` is a device-emitted event, so it carries **no response / code** — the command response-code table does not apply, and no response code is defined for any event. The reader always emits it on a connection-state change; it needs no `config events` toggle. (The contract names this message `epConnectionEvent`, a cosmetic mismatch with the event name — the on-the-wire payload is unaffected.)

`mqttConnEVT` is **read-only telemetry** with no credentials (`deviceSerialNo` is an identifier; `apiVersion` / `mqttVersion` are version values). Use the `rfd`-topic connection events for a reliable presence/liveness signal — the `DISCONNECTED` Last Will tells you a device dropped even when the device itself cannot. For the concept-level view, see [MQTT Connectivity](#) and [Broker Connection](#).

Related:  [alerts](#) ·  [heartBeatEVT](#) ·  [dataEVT](#) ·  [Events](#) ·  [MQTT Connectivity](#) ·  [Broker Connection](#)